# MoE architecture choices: Qwen3, OLMoE and Marin

Convenient widths, actual attention dimensions, expert geometry and replication targets

## Powers of two are a convenient candidate set

Yes: starting from widths 512, 1024, 2048 and 4096 reduces the search and often gives convenient matrix/sharding shapes. It is an engineering heuristic, not an architectural requirement or evidence of a loss optimum. Multiples of 128/256 such as 768, 1280, 1536, 3072 and 6144 are also common. The relevant constraints are the actual projection shapes, attention head geometry, tensor/expert sharding, grouped-GEMM alignment, memory and measured tokens/s.

For our decision, enumerate a short set of aligned widths, derive several feasible depths, and measure them on the intended hardware. A derived depth makes a family reproducible; it does not prove that depth is optimal. See the architecture evidence guide and Marin's optimizer-search walkthrough.

## Concrete configurations, with version and scope

The local configuration audit pins three official HF configuration revisions and Marin commit `1d274b52c636b6f2a9177b1d4c14b2f74795a8a6`. These are **reported/released configurations**, not results of our shape search.

| Configuration | Residual width | Layers | Q / KV heads | Head dimension | Routed experts / top-k | Routed expert FFN width | Shared experts |
|---|---|---|---|---|---|---|---|
| Qwen3-30B-A3B | 2048 | 48 | 32 / 4 | 128 | 128 / 8 | 768 | 0 |
| Qwen3-235B-A22B | 4096 | 94 | 64 / 4 | 128 | 128 / 8 | 1536 | 0 |
| OLMoE-1B-7B-0924 | 2048 | 16 | 16 / 16 | 128, inferred from width/heads | 64 / 8 | 1024 | 0 |
| Marin EP hero, inspected code | 6144 | 48 | 48 / 12 local, 48 / 6 global | 128 | 384 / 8 | 3072 in a 3072-dimensional latent | 2, width 3072 |

Qwen sources: 30B configuration, 235B configuration, and Qwen3 report, §2/Table 2. OLMoE sources: configuration, report. Marin source: EP heuristic/HERO_MODEL.

OLMoE is the MoE comparison here: OLMo 2/3 dense models are separate families, described in the OLMo effort guide. Likewise, Marin's public dense models, dense Delphi, May MoE recipe and later EP hero are not one interchangeable architecture. The EP hero row describes inspected experiment/production code, not an assertion that we downloaded a released 535B model checkpoint.

## A crucial Qwen3 detail: residual width is not Q projection width

In the released Qwen3-30B-A3B config,

$$d_{\text{residual}} = 2048, \qquad h_Q d_h = 32 \times 128 = 4096.$$

For the 235B model, $d_{\text{residual}} = 4096$ while $h_Q d_h = 8192$. Thus the common shortcut $d = h_Q d_h$ is false for these models. The output attention projection maps the concatenated query-head space back into the residual stream. Our earlier analytical candidate counter enforces that shortcut **for its own chosen family**; it cannot directly count Qwen3 without generalization.

Let $d$ be residual width, $a = h_Q d_h$ query/output attention width and $b = h_{KV} d_h$ KV projection width. Ignoring biases, the attention projection parameter count is

$$P_{\text{attn}} = da + db + db + ad = 2d(a + b).$$

For standard every-layer SwiGLU MoE with expert FFN width $f$, $E$ experts, top-$k$ routing, $n$ layers and no shared experts,

$$P_{\text{experts}} = nE(3df), \qquad P_{\text{executed experts}} = nk(3df).$$

Add attention, routers, norms, embeddings/head and any latent/shared projections separately. A sparsity fraction $k/E$ describes expert selection, not the fraction of the **whole model** executed. Layer repetition increases executions without creating new stored parameters; none of the four rows above should be presumed recurrent just because our earlier candidate proposed a loop.

The Qwen3 configs have `moe_intermediate_size` distinct from `intermediate_size`; use the former for routed expert counts. They use normalized selected expert probabilities (`norm_topk_prob=true`), while the inspected OLMoE config has it false. This affects computation and gradients and is not a cosmetic flag.

## What Qwen3's choices imply

Qwen3 uses fine-grained experts, no shared expert, RMSNorm/SwiGLU, QK normalization and GQA. The report describes global-batch load balancing. Both MoE sizes keep 128 experts/top-8 while increasing depth and projection widths, but that is one successful family progression, not evidence that expert count should always remain fixed.

The report's tokenizer vocabulary and the padded HF matrix vocabulary are different quantities: the inspected configs use 151,936 embedding rows. The configs also store `max_position_embeddings=40960`, whereas the report advertises 128K context after long-context training/extension. State which stage/configuration is being copied rather than treating one field as the entire context-training history.

## OLMoE is a simpler open control

OLMoE's 16-layer, width-2048 model has 64 small experts, top-8 routing, full multi-head attention and no shared experts. Its name distinguishes approximately 1B activated from 7B total parameters. The paper supplies architecture and training ablations, open corpus information and optimizer settings; it is a strong reference for a transparent standard-MoE control.

That does not make its width/depth ratio optimal at our much longer horizon. Copying its QK-norm, initialization, routing and loss reductions faithfully is useful; selecting expert count/depth or LR requires target-like experiments. Compare total and active counts under the same embedding/head convention.

## Marin has several MoE recipes, not one

The June efficiency report starts with **64 routed experts/top-4**, AdamH, quantile balancing, router/logit z-losses and gated components. It then reports ablations increasing to 256 experts and changing to MuonH. The dense-to-MoE transition was tested at $10^{23}$ FLOPs; later changes were measured against MoE V1, with the stacked recipe tested at a smaller budget. Those results do not isolate every interaction at hero scale.

The later August LR sweep used **128/top-4 plus two shared experts**, hybrid local/global attention and SConv. The inspected EP hero instead uses **384/top-8**, a half-width latent expert space, two shared experts, gated norms/attention, QB routing and a ragged all-to-all implementation. These are materially different configurations.

At the inspected commit, `launch_scaling_ladder.py` lists a hero-shape ladder:

| Rung | Width | Depth from `SMALL_SHAPES` / hero | Reported active / total in launcher comments | Target tokens in comments |
|---|---|---|---|---|
| d768 | 768 | 8 | 61M / 1.6B | 48B |
| d1024 | 1024 | 12 | 162M / 4.0B | 128B |
| d1536 | 1536 | 16 | 481M / 11.5B | 381B |
| d2048 | 2048 | 22 | 1.2B / 27.7B | 926B |
| d6144 | 6144 | 48 | 23B / 535B | ~18T |

These parameter/token figures are approximate **source-comment accounting**, not independently instantiated tensor counts here. The launcher targets **791 tokens per active parameter**. Do not compare that directly with our 200-token-per-**stored**-parameter requirement. For perspective, the approximate 535B/23B counts convert791 active-TPP into only about 34 stored-TPP; our200 stored-TPP preference is a substantially longer regime relative to capacity. The depth changes, including22 layers at d2048, also differ from an older issue's sweep table. Follow the executable pinned configuration for reproduction and record such drift.

The current hero's context history changes from 4K to 16K and keeps tokens per rack/update constant, reducing sequence batch accordingly. Source comments describe a running recipe with restarts and interventions; it is not a pristine single immutable run whose every final choice was fixed before training.

## How we should use these examples

1. Select the reproduction target: a historical Marin recipe with accessible inputs, or a new implementation of its principles. Record the exact commit, optimizer, attention/router/expert geometry and data release.
2. Establish a standard non-looped baseline first. Cyber data and recurrent depth are separate ablations after that baseline works.
3. Enumerate aligned widths such as 1024/1536/2048 and a small depth set. Check projection dimensions, head divisibility and expert/TP sharding; do not insist on powers of two or force $h_Q d_h = d$.
4. Count all stored tensors and executed FLOPs with the actual model, then profile useful tokens/s and serving memory/latency.
5. Tune and validate at the intended overtraining horizon before selecting the hero shape. Use the Marin search procedure, not only an architecture table.

No configuration in this comparison is a measured winner on our 96-H100 allocation.